AI Forward Deployed Engineer

The AI Architect & FDE Course



Week 1 Day 4

3 weeks to Forward Deployed Engineer

Theory Practical Projects

Week 3: Become an AI FDE

- AI Solutions Architect
- Security, Guardrails, API Gateways
- Enterprise Context Layer & Integrations
- Production Deployment
- Capstone

Week 2: Become an AI Architect

- Architecture, Evals, Observability
- Agentic RAG, MCP, Memory
- Durable State & Human in the Loop
- Agent Teams, Swarms, Fleet
- Self-Improvement, RSI

Week 1: Become an AI Engineer

- Path to FDE
- Frontier LLMs & Agents
- Coding Agents & ADLC
- Open-Source, Local, Token Cost
- Harness Engineering

Taking stock

You can already:

- [x] Describe the role of the AI Forward Deployed Engineer
- [x] Define an agent and a harness, and build an agent with LangChain
- [x] Follow an ADLC to deliver code with controls
- [x] Accelerate delivery using Coding Agents while staying accountable

Today has 30 slides and a lab! But the goal is awareness:

- [ ] Differentiate between proprietary and open-weight models
- [ ] Intuition for quantization and mixture of experts
- [ ] Aware of how to deploy and run inference on LLMs
- [ ] Hands-on experience running models locally

Recap

Coding agents and the ADLC on Day 3



The coding agent landscape

IDEs, plugins and terminal agents such as Claude Code and OpenCode



Working with Claude Code

AGENTS.md, permissions, a plan first and a review of what it wrote



The ADLC

Building software with agents while staying accountable for the results



The pilot

Claude Code planned and built the three pieces of UnderCover

For today, we're putting the pilot to one side - it will return tomorrow!

Setting the scene



Inference

Using a trained model. It reads an input sequence and predicts the output one token at a time, feeding each token back in (the auto-regression trick)



GPU

A graphics card built to run huge numbers of matrix calculations in parallel. Great for drawing lots of polygons 60 times a second and, it turns out, great for LLM inference.



GPU memory

The memory the GPU works from. On a PC it is usually separate from the CPU memory and called VRAM. On a Mac the CPU and GPU share one pool, called unified memory.



Bandwidth

How many GB a second the GPU can read from its memory. It usually decides how fast a model writes its answer.

Proprietary and open models

Two completely different approaches to LLMs



PAID FRONTIER

Proprietary models

Only the lab has the weights. You call GPT-6, Claude or Gemini through the lab's own API or a cloud partner like Amazon Bedrock.



Open-source

Weights, training data and training code are all published

K2 Horizon from MBZUAI



Open-weight

The weights are free to use. The training data and methods are private.

Qwen 3.5, Gemma 4, DeepSeek V4.1



Restricted

Open weights under a licence that limits commercial use

Mistral Medium 3.5, Kimi K3

All the last row are "Open" models and people often loosely say "Open-Source"

The case for open-weight models

Most of the reasons come down to privacy and control



Why open-weight

Privacy
On your own hardware, customer data never leaves your network

Independence
On a cloud host like Groq you rely on the host instead of the lab

Control
Pin an exact version, fine-tune it and run it air-gapped



What it costs you

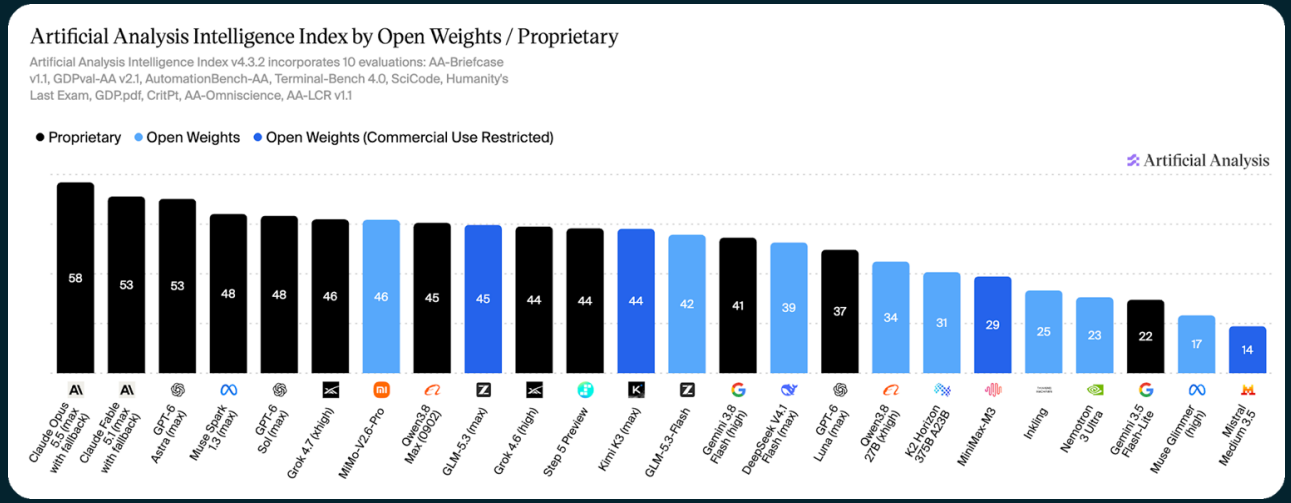
Hardware
Buying or renting GPUs means paying for them even when idle

Operations
Updates, monitoring, scaling and security are yours to run

Smaller models
A less capable agent can leak data, take the wrong action or run up costs

It's not necessarily the case that Open-Source models are cheaper in practice; Eg GPT Luna is at the pareto frontier

How open-weight models compare



Now go and look at your version!

The labs that dominate the news

The most important labs for me now

Frontier and proprietary

OpenAI
GPT Astra, Sol, Terra and Luna

Anthropic
Claude Fable, Opus, Sonnet and Haiku

Google
Gemini 3.x Pro, Flash and Flash-Lite

xAI
Grok 4.7

Open-weight

Alibaba Cloud
Qwen 3.8 and Qwen 3.6

Moonshot AI
Kimi K3

Z.ai
GLM 5.3 and GLM 5.3 Flash

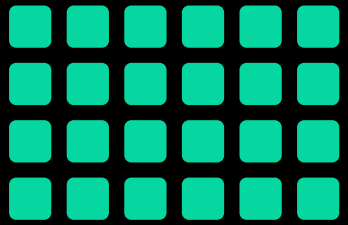
Xiaomi
MiMo V2.6 Pro and Flash

Honorable mention: Meta
Muse Spark is proprietary while Llama 4 and Muse Glimmer are open-weight

Dense and Mixture of Experts models

An MoE model uses a small share of its weights for each token

Dense



Every weight works on every token

Qwen 3.6 27B

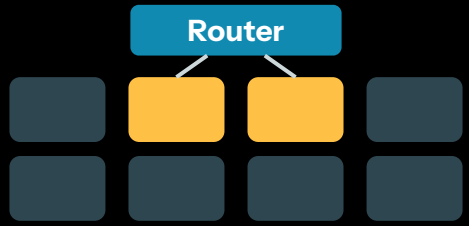
27B parameters, all active

No experts

18 GB · 17 tokens/s on an M1 Max

Intelligence Index 21

Mixture of Experts



A router picks a few experts

Qwen 3.6 35B-A3B

35B parameters, 3B active per token

256 experts, 8 picked plus 1 shared

23 GB · 61 tokens/s on an M1 Max

Intelligence Index 18

How the router works

The router is part of the model and learns its choices in training



Qwen 3.6 35B-A3B repeats this in each of its 40 layers for every token

The router

A small learned layer in each MoE layer. It all runs in one pass with no extra LLM calls.

The experts

They pick up small jobs like punctuation, numbers, names or closing brackets in LaTeX

By contrast..

OpenRouter routes to different providers. Agentic Workflows can make separate LLM calls.

Gemma 4 26B A4B picks 8 of 128 experts, DeepSeek V4.1 Flash 6 of 384 and Kimi K3 16 of 896

Dense or Mixture of Experts?

Comparing the Qwen 3.6 alternatives



Choose dense

Quality per GB
The 27B beats the 35B-A3B on every benchmark they share, in a smaller file

Limited memory
On a 24 GB GPU the 27B leaves room for context and the MoE barely fits

Fine-tuning
A LoRA on Qwen 3.5 27B needs 56 GB against 74 GB for the 35B-A3B



Choose MoE

Speed
3.7 times faster on a Mac and 4.6 times on a 16 GB NVIDIA card

Serving cost
Alibaba charges 38% less per output token for the 35B-A3B

Big unified memory
Suits a Mac Studio or DGX Spark with lots of memory and modest bandwidth

An MoE model needs all GPU memory available to fit the full model, but runs fast

Quantization

Storing each weight in fewer bits shrinks the model and speeds it up

```
GB = billions of parameters x bits per weight / 8
```

Qwen 3.5 27B at Q4_K_M: 27.8 x 4.9 / 8 = 17 GB, plus room for the context

Llama 3.1 8B in llama.cpp: 29 tok/s at 16 bits, 51 at Q8_0 and 72 at Q4_K_M

Common precisions

Precision	Bits	Where you see it
BF16	16	What labs publish
FP8	8	Most providers
Q8_0	8.5	Near lossless GGUF
Q4_K_M	4.9	Ollama's default
NVFP4	4.5	NVIDIA 4-bit float
MXFP4	4.25	gpt-oss, Kimi K3

8 bits lose little

Llama 3.1 kept 99.75% of its score on average and 99.9% on coding

4 bits - you feel the change

Llama 3.1 kept 99.36% of its score on average and 98.9% on coding

Quantization-aware training

The model is trained with the 4-bit rounding simulated. It loses less than a model rounded after training.

Sources: llama.cpp; Kurtic et al., 500,000 evaluations, arXiv 2411.02355; Google on Gemma 4 QAT

File formats

(So that you recognize these when you see them)

safetensors
From Hugging Face

The format most labs use to publish BF16 or FP8 weights
Read by vLLM, SGLang and Hugging Face Transformers
`Qwen/Qwen3.5-27B-FP8`

GGUF
From llama.cpp

One file with the weights, tokenizer and chat template
Read by llama.cpp, Ollama and LM Studio
`Qwen3.5-27B-Q4_K_M.gguf` `qwen3.5:27b`

MLX
From Apple

Safetensors files in the layout that Apple's MLX expects
Read by mlx-lm, LM Studio and Ollama on a Mac
`mlx-community/Qwen3.5-27B-4bit` `qwen3.5:27b-mlx`

How GGUF packs the weights

GGUF is the file format created by the llama.cpp team

The weights can be packed in different ways

Type	Bits	How the weights are packed
Q8_0	8.5	The original 8-bit scheme, close to lossless
Q6_K	6.6	A 6-bit k-quant, almost as good as Q8_0
Q5_K_M	5.7	A 5-bit k-quant with the medium mix
Q4_K_M	4.9	A 4-bit k-quant with key tensors at 6 bits
Q4_0	4.6	The original 4-bit scheme, now outdated
IQ4_XS	4.3	A 4-bit i-quant tuned on sample text
Q2_K	3.2	A 2-bit k-quant that loses a lot of quality

Q4	About 4 bits for most of the weights
_0	The original scheme, one scale for each block of 32 weights
K	k-quants, with a scale for each block of 32 inside blocks of 256
S M L	Small, medium or large: how many sensitive tensors get more bits
IQ	i-quants, which use sample text to find the weights that matter most

Reading a model's name

It usually tells you the parameter count, how many are active, format

`nemotron-3-ultra-550b-a55b`	NVIDIA Nemotron 3 Ultra: 550B in total, 55B active per token
`gemma-4-26b-a4b-it`	Google Gemma 4: 26B in total, 4B active, instruction tuned (it) for chat
`gemma4:e4b-it-qat`	Gemma 4 in Ollama: 4B effective parameters, quantization-aware trained
`qwen3.5:4b-q4_K_M`	Qwen 3.5 with 4B parameters, quantized to 4 bits in the K_M format of llama.cpp
`google/gemma-4-31b-it:free`	On OpenRouter the maker comes first and :free is the free variant with daily limits

Total size **Active per token** **Tuning and variant** **Quantization**

Ollama, LM Studio, llama.cpp, vLLM

From one developer's laptop to a GPU server shared by many users

For one person

`ollama`
Open-source command line and desktop app
Mac, PC and Linux

`LM Studio`
A free closed-source desktop app
Mac, PC and Linux

Under the hood

`llama.cpp`
The original C/C++ engine inside both apps
CPU and any GPU

`mlx-lm`
Apple's own engine for Macs
Apple silicon only

For many users

`vllm`
The standard for serving many users
NVIDIA and AMD GPUs

`sglang`
vLLM's main rival for serving at scale
NVIDIA and AMD GPUs

Typically access via an OpenAI compatible API

vLLM runs on a Mac only through the community vllm-metal plugin

One model, many variants

“Distillation”

A smaller, ‘distilled’ version of a flagship model with fewer parameters; typically significantly smaller but far less intelligent

Ollama's tags for two of today's local models

Tag	gemma4:e4b	qwen3.5:4b	What it is
bf16	16 GB	9.3 GB	The weights as published
q8_0	12 GB	5.3 GB	GGUF, 8 bits, near lossless
q4_K_M	9.6 GB	3.4 GB	GGUF, 4 bits, the default
qat	6.1 GB	none	Google's QAT file, 4 bits
nvfp4	9.5 GB	4.0 GB	NVIDIA's 4-bit float, on MLX
mxfp8	12 GB	5.6 GB	8-bit microscaling, on MLX

What fits on your machine

Based on GPU or Unified memory for a 4 bit model

8 GB laptop	Qwen 3.5 4B (3.4 GB) or Gemma 4 E2B QAT (4.3 GB)
16 GB	Gemma 4 E4B (9.6 GB) or Qwen 3.5 9B (6.6 GB)
32 GB	Qwen 3.5 27B (17 GB) or Gemma 4 26B A4B (19 GB)
128 GB	Qwen 3.5 122B A10B (81 GB) or Nemotron 3 Super (87 GB)
256 GB	Qwen 3.5 397B A17B (214 GB)
Data center	DeepSeek V4.1 Flash (510 GB) or Kimi K3 (1.5 TB)

llmfit
Ranks every model by how well it runs on your machine

```
uv tool install llmfit
llmfit
```

An 8 GB Mac gives the GPU about 5 GB.
A laptop with no GPU runs Gemma 4 E4B at 5 to 8 tokens a second.

Memory bandwidth sets the speed

Each new token reads every active weight from memory once

```
top speed = bandwidth (GB/s) / model size (GB)
```

Qwen 3.8 27B at 4 bits on a DGX Spark: 273 GB/s / 17 GB = 16 tokens a second at most

Machine	Memory	Bandwidth	Top speed	Measured
DGX Spark	128 GB	273 GB/s	16 tok/s	12 tok/s
M1 Max	64 GB	400 GB/s	24 tok/s	17 tok/s
M5 Ultra	256 GB	1.2 TB/s	70 tok/s	48 tok/s
RTX 5090	32 GB	1.8 TB/s	105 tok/s	75 tok/s



MoE models read only their active weights

Qwen 3.6 35B-A3B reads about 2 GB a token and runs at 61 tok/s on an M1 Max



Reading the prompt depends on compute

NVIDIA GPUs usually get through a long prompt faster than a Mac

Top speed divides the bandwidth by 17 GB. Measured with Qwen 3.8 27B by Hardware Corner and MacStories, and on the M1 Max with Qwen 3.6 27B in mlx-lm

Four recipes for an inference server

If you have the budget and the need to run models yourself

	Price	Memory	Bandwidth	Biggest model	Speed
Mac Studio, M5 Ultra Most memory in one box	**$9,499** from $5,499	**256 GB** unified	**1.2 TB/s**	**400B** Qwen 3.5 397B	**48 tok/s** measured
NVIDIA DGX Spark Suited to MoE models	**$4,699** list price	**128 GB** unified	**273 GB/s**	**200B** NVIDIA's figure	**12 tok/s** measured
RTX 5090 PC tower 700 to 900 W at the wall	**$8,500** to $9,500 built	**32 GB** VRAM	**1.8 TB/s**	**35B** Qwen 3.6 35B	**75 tok/s** measured
Runpod Rent an RTX PRO 6000	**$2.09/h** $1,526/month	**96 GB** VRAM	**1.8 TB/s**	**120B** Qwen 3.5 122B	**75 tok/s** estimate

Prices in US dollars in September 2026, raised by the memory shortage
Sizes and speeds at 4 bits. Qwen 3.8 27B speeds from Hardware Corner (llama.cpp) and MacStories (MLX)
Check prices at apple.com/mac-studio · nvidia.com/dgx-spark · pcpartpicker.com · runpod.io/pricing

Now to install Ollama

Download and install it from ollama.com

Then in a terminal

`ollama serve`	Starts the server, if it isn't running already
`ollama pull gemma4:e4b`	Downloads a model
`ollama list`	Lists the models you've downloaded

Visit `localhost:11434` in your browser to see “Ollama is running”

Two distractions to ignore

Ollama Cloud A newer service that runs the models on Ollama's servers

`ollama` **on its own** Opens a menu to set up Claude Code, OpenClaw and others

OpenRouter traps

OpenRouter can send the same model to dozens of providers

Model	Providers	Precisions	Input $/M
GLM 5.3 Flash	33	fp4, nvfp4, fp8	0.045 to 0.55
Kimi K3	19	fp4, mxfp4, fp8, bf16	1.00 to 4.50



In my test, Kimi K3 came in under its list price

In our test run, 13 of the 20 calls went to a cheaper provider serving it at fp4



But DeepSeek V4.1 Flash cost three times its list price

In our test run its calls went to more expensive providers than the $0.035 one



A best practice is to test and then pin the providers

For repeatable results

What differs between providers

Features, precision and limits change from one provider to the next

Provider	Precision	Structured	Input $/M	Max output
InferenceNet	fp4	yes	0.045	128,000
Io Net	fp8	yes	0.135	65,536
Z.AI	fp8	no	0.15	131,072
CoreWeave	nvfp4	yes	0.15	943,718
Wafer	unknown	yes	0.75	943,718

Structured outputs failed on some providers

In our runs Darkbloom sent back an empty answer and Together rejected the request

Some providers ignored our settings

In our runs Google Vertex ignored the reasoning setting and once ran for 19 minutes

How to choose the provider yourself



Pin the providers on OpenRouter

Test a few providers with your real prompt and list only the ones that pass

```
extra_body={"provider": {
    "order": ["parasail", "baseten"],
    "quantizations": ["fp8"],
    "allow_fallbacks": False, "require_parameters": True}}
```



Go direct to the provider

Cerebras serves Qwen 3.8 27B at about 1,850 tokens a second

```
client = OpenAI(base_url="https://api.cerebras.ai/v1")
client.chat.completions.create(
    model="qwen-3.8-27b", messages=messages)
```

Going direct also avoids OpenRouter's 5.5% fee on card top-ups

How token pricing works

Output tokens cost several times more than input tokens



What you pay for

Input tokens
Everything you send to the model

Cached input
A repeated prefix at a lower price

Output tokens
Everything the model writes back

Reasoning tokens
Billed as output even when hidden



With a coding agent

`/compact` and `/clear` often
Every turn resends the whole context

A short AGENTS.md
It is part of every request

Avoid resuming a full session
Start fresh or compact first

Compact within the hour
A Claude plan caches for an hour

A prompt that gives accurate answers usually matters more than saving input tokens

Working out the cost per claim

One UnderCover claim on GLM 5.3 Flash with today's trimmed

```
Input     1,110 tokens  x  $0.15 per million  =  $0.00017
Output      138 tokens  x  $0.50 per million  =  $0.00007
                                  One claim   =  $0.00024
                          10,000 claims a month  =  $2.40
```

What moves the cost



The prompt

Today's prompt is less than half the size of the one on Day 2



The reasoning

Thinking adds output tokens, which cost more

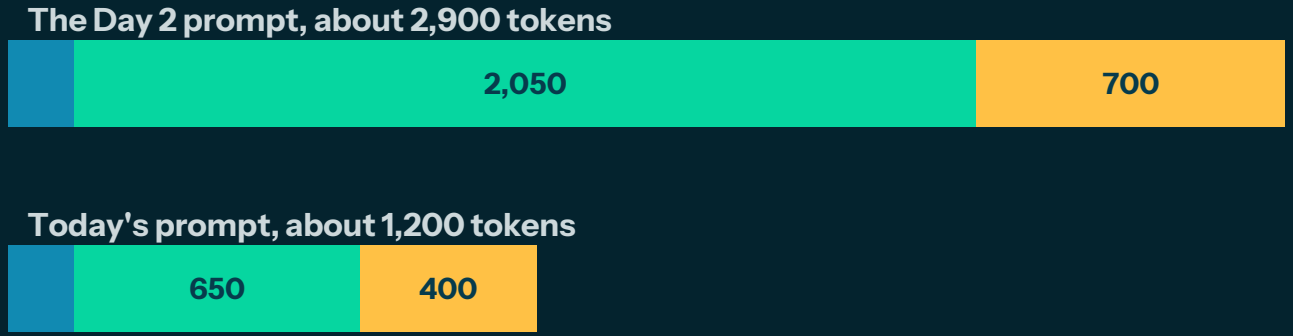


The price

Set by the model, and by the provider serving it

Trimming the prompt

Today's prompt keeps only the wording that applies to the claim



Left out today: the sections for other benefits, the definitions, the ids and the text of the documents

The nine models in today's lab

Six hosted on OpenRouter and three on your own machine

Model	Total / active	Runs on	$/M in / out
MiMo V2.6 Pro	1.02T / 42B	OpenRouter	0.435 / 0.87
GLM 5.3 Flash	320B / 18B	OpenRouter	0.15 / 0.50
Kimi K3	2.8T / 104B	OpenRouter	3.00 / 15.00
DeepSeek V4.1 Flash	552B / 16B	OpenRouter	0.035 / 0.29
Nemotron 3 Ultra	550B / 55B	OpenRouter	0.60 / 2.40
Mistral Medium 3.5	128B dense	OpenRouter	1.50 / 7.50
Gemma 4 E4B	4.5B effective	Ollama 9.6 GB	free
Qwen 3.5 4B	4.7B dense	Ollama 3.4 GB	free
Gemma 4 E2B QAT	2B effective	Ollama 4.3 GB	free

And one surprise optional extra

Today's lab

In `labs/week1day4.ipynb` and free twin and CPU twin

1. **The data and a smaller prompt**
   The 20 claims that need no claims manual, and a prompt half the size

2. **Six open-weight models on OpenRouter**
   One call at a time, at the lowest reasoning level each offers

3. **Three models on your computer**
   Qwen 3.5 4B and Gemma 4 E2B and E4B through Ollama

4. **Accuracy, cost and speed side by side**
   Three charts drawn from your own runs

3 weeks to Forward Deployed Engineer

Theory Practical Projects

Week 3: Become an AI FDE

AI Solutions Architect	Security, Guardrails, API Gateways	Enterprise Context Layer & Integrations	Production Deployment	Capstone

Week 2: Become an AI Architect

Architecture, Evals, Observability	Agentic RAG, MCP, Memory	Durable State & Human in the Loop	Agent Teams, Swarms, Fleet	Self-Improvement, RSI

Week 1: Become an AI Engineer

Path to FDE	Frontier LLMs & Agents	Coding Agents & ADLC	Open-Source, Local, Token Cost	Harness Engineering

AI Forward Deployed Engineer

The AI Architect & FDE Course



27% Complete